

Souborové systémy a streamy

Souborový systém

- **souborový systém** = způsob, jakým operační systém organizuje data na disku
- určuje:
 - jak jsou soubory uložené
 - jak jsou pojmenované
 - jak jsou uspořádané do složek
 - jak se k nim přistupuje
 - jaká mají oprávnění

Příklady souborových systémů:

- **NTFS** — často Windows
- **FAT32** — starší, flash disky
- **exFAT** — flash disky, externí disky
- **ext4** — často Linux

Struktura souborového systému

Hierarchická struktura

- většina souborových systémů má stromovou strukturu
- nahoře je kořen
- pod ním jsou složky a soubory

```
kořen
├── složka
│   ├── soubor.txt
│   └── obrazek.png
└── program.exe
```

Kořen

- **kořen** = nejvyšší místo ve stromové struktuře
- od něj začíná cesta k souborům

Příklady:

```
C:\
```

/

- Windows často používá například `C:\`
- Linux a macOS používají `/`

Adresář / složka

- **adresář** = složka
- slouží k organizaci souborů
- může obsahovat:
 - soubory
 - další složky

Příklad:

```
Dokumenty
├── skola
├── obrazky
└── poznamky.txt
```

Soubor

- **soubor** = pojmenovaná jednotka dat
- obsahuje konkrétní data
- může to být například:
 - text
 - obrázek
 - zvuk
 - video
 - program
 - binární data

Příklady:

- `poznamky.txt`
- `obrazek.png`
- `program.exe`
- `data.bin`

Cesty k souborům

Absolutní cesta

- začíná od kořene souborového systému

- jednoznačně určuje umístění souboru

Příklad:

```
C:\Users\Jan\Documents\data.txt
```

Příklad Linux:

```
/home/jan/Documents/data.txt
```

Relativní cesta

- vychází z aktuální složky
- je kratší
- závisí na tom, odkud program běží

Příklad:

```
Data/vstup.txt
```

```
../vystup.txt
```

Rozdíly mezi operačními systémy

Windows

- oddělovač cest je většinou `\`
- používá například disky `C:\`, `D:\`
- většinou nerozlišuje velikost písmen v názvech souborů

Příklad:

```
C:\Windows\System32
```

Linux / macOS

- oddělovač cest je `/`
- kořen je `/`
- obvykle rozlišuje velikost písmen

Příklad:

```
/dev/null
```

Poznámka:

- `soubor.txt` a `Soubor.txt` mohou být na Linuxu dva různé soubory
- na Windows se často berou jako stejný název

Path.Combine

- v C# je vhodné používat `Path.Combine`
- složí cestu správně podle operačního systému
- pomáhá s multiplatformností

Příklad:

```
string cesta = Path.Combine("Data", "vstup.txt");
```

Práce se soubory

I/O operace

- práce se soubory patří mezi **I/O operace**
- I/O = Input / Output
- tedy vstup a výstup dat
- soubory můžeme:
 - číst
 - zapisovat
 - vytvářet
 - mazat
 - kopírovat
 - přesouvat
 - procházet

Rizika při práci se soubory

- soubor nemusí existovat
- složka nemusí existovat
- program nemusí mít oprávnění
- soubor může používat jiný proces
- disk může být plný
- cesta může být špatně zapsaná
- může dojít k chybě při čtení nebo zápisu

Proto se často používá:

- `try`

- `catch`
- `finally`
- `using`

Namespace `System.IO`

- V C# se pro práci se soubory používá hlavně namespace:

```
using System.IO;
```

Důležité třídy:

Třída	Použití
<code>File</code>	práce se soubory
<code>Directory</code>	práce se složkami
<code>Path</code>	práce s cestami
<code>FileInfo</code>	informace o souboru
<code>DirectoryInfo</code>	informace o složce
<code>FileStream</code>	proud bajtů
<code>StreamReader</code>	čtení textu
<code>StreamWriter</code>	zápis textu
<code>BinaryReader</code>	čtení binárních dat
<code>BinaryWriter</code>	zápis binárních dat

Textové soubory

Co je textový soubor

- obsahuje znaky čitelné pro člověka
- používá znakové kódování
- například:
 - UTF-8
 - ASCII
 - Windows-1250
- často se čte po řádcích

Příklady:

- `.txt`
- `.csv`

- .json
- .xml
- .html

Pozor:

- přípona souboru sama o sobě nezaručuje obsah
- soubor `data.txt` může obsahovat binární data
- skutečný typ souboru může určovat obsah nebo hlavička souboru

Čtení celého textového souboru

```
string text = File.ReadAllText("data.txt");

Console.WriteLine(text);
```

Čtení textového souboru po řádcích

```
string[] radky = File.ReadAllLines("data.txt");

foreach (string radek in radky)
{
    Console.WriteLine(radek);
}
```

Zápis textu do souboru

```
File.WriteAllText("vystup.txt", "Ahoj světe");
```

- pokud soubor neexistuje, vytvoří se
- pokud existuje, přepíše se

Přidání textu na konec souboru

```
File.AppendAllText("vystup.txt", "\nDalší řádek");
```

- text se přidá za existující obsah

Streamy

Co je stream

- **stream** = datový proud
- abstrakce pro postupné čtení nebo zápis dat

- data netečou najednou, ale po částech
- dá se představit jako potrubí, kterým tečou data
- používá se u:
 - souborů
 - síťové komunikace
 - paměti
 - komprese

Proč se používají streamy

- nemusíme načíst celý soubor do paměti
- vhodné pro velké soubory
- umožňují číst a zapisovat postupně
- umí pracovat s aktuální pozicí v souboru

Pozice ve streamu

- stream si může pamatovat aktuální pozici
- tato pozice říká, kde právě čteme nebo zapisujeme
- v C# se často používá vlastnost `Position`

Příklad:

```
stream.Position = 0;
```

- nastaví pozici na začátek streamu

FileStream

- `FileStream` je základní proud bajtů
- pracuje přímo s bajty
- hodí se pro binární data
- lze ho použít i jako základ pro textové čtení a zápis

Příklad:

```
using FileStream fs = new FileStream("data.bin", FileMode.OpenOrCreate);
```

StreamReader a StreamWriter

StreamReader

- používá se pro čtení textu ze streamu
- převádí bajty na znaky

- typicky se používá s textovými soubory

Příklad:

```
using StreamReader reader = new StreamReader("data.txt");

string obsah = reader.ReadToEnd();

Console.WriteLine(obsah);
```

StreamWriter

- používá se pro zápis textu do streamu
- převádí znaky na bajty
- typicky se používá s textovými soubory

Příklad:

```
using StreamWriter writer = new StreamWriter("vystup.txt");

writer.WriteLine("První řádek");
writer.WriteLine("Druhý řádek");
```

Binární soubory

Co je binární soubor

- obsahuje data v binární podobě
- pro člověka obvykle není přímo čitelný
- data se čtou jako bajty
- často se čte po blocích pomocí bufferu

Příklady:

- .png
- .jpg
- .mp3
- .exe
- .pdf
- .zip
- .bin

Čtení binárního souboru jako bajty

```
byte[] data = File.ReadAllBytes("obrazek.png");

Console.WriteLine(data.Length);
```

Zápis binárního souboru

```
byte[] data = { 1, 2, 3, 4, 5 };

File.WriteAllBytes("data.bin", data);
```

Čtení binárního souboru po blocích

```
byte[] buffer = new byte[1024];

using FileStream fs = new FileStream("data.bin", FileMode.Open);

int nacteno;

while ((nacteno = fs.Read(buffer, 0, buffer.Length)) > 0)
{
    Console.WriteLine("Načteno bajtů: " + nacteno);
}
```

- čte se po blocích
- vhodné pro větší soubory
- není nutné načíst celý soubor najednou

BinaryReader a BinaryWriter

BinaryReader

- umožňuje číst binární data ve formě konkrétních typů
- například:
 - `int`
 - `double`
 - `bool`
 - `string`

Příklad:

```
using BinaryReader reader = new BinaryReader(File.Open("data.bin",
FileMode.Open));
```

```
int cislo = reader.ReadInt32();
double hodnota = reader.ReadDouble();
```

BinaryWriter

- umožňuje zapisovat binární data ve formě konkrétních typů

Příklad:

```
using BinaryWriter writer = new BinaryWriter(File.Open("data.bin",
    FileMode.Create));

writer.Write(123);
writer.Write(45.6);
writer.Write(true);
```

using

- `using` zajistí automatické zavření souboru nebo streamu
- po skončení bloku se objekt uvolní
- používá se hlavně u souborů a streamů
- zabraňuje tomu, aby soubor zůstal otevřený

Příklad:

```
using StreamReader reader = new StreamReader("data.txt");

string text = reader.ReadToEnd();
```

Výjimky při práci se soubory

Proč je řešit

- práce se soubory může selhat
- program by neměl spadnout bez vysvětlení
- chyby je vhodné ošetřit pomocí `try-catch`

Příklad:

```
try
{
    string text = File.ReadAllText("data.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
}
```

```
        Console.WriteLine("Soubor nebyl nalezen.");
    }
    catch (UnauthorizedAccessException)
    {
        Console.WriteLine("Nemáš oprávnění k souboru.");
    }
    catch (IOException)
    {
        Console.WriteLine("Chyba při práci se souborem.");
    }
}
```

Práce se složkami

Zjištění souborů ve složce

```
string[] soubory = Directory.GetFiles("Data");

foreach (string soubor in soubory)
{
    Console.WriteLine(soubor);
}
```

Zjištění složek ve složce

```
string[] slozky = Directory.GetDirectories("Data");

foreach (string slozka in slozky)
{
    Console.WriteLine(slozka);
}
```

Rekurzivní procházení složky

```
void ProjdiSlozku(string cesta)
{
    foreach (string soubor in Directory.GetFiles(cesta))
    {
        Console.WriteLine("Soubor: " + soubor);
    }

    foreach (string slozka in Directory.GetDirectories(cesta))
    {
        Console.WriteLine("Složka: " + slozka);
        ProjdiSlozku(slozka);
    }
}
```

Praktické ukázky

1. Vypsání obsahu textového souboru

```
try
{
    string text = File.ReadAllText("data.txt");
    Console.WriteLine(text);
}
catch (Exception ex)
{
    Console.WriteLine("Chyba: " + ex.Message);
}
```

2. Zápis textu do souboru

```
try
{
    File.WriteAllText("vystup.txt", "Ahoj světe");
}
catch (Exception ex)
{
    Console.WriteLine("Chyba: " + ex.Message);
}
```

3. Čtení souboru po řádcích

```
try
{
    string[] radky = File.ReadAllLines("data.txt");

    foreach (string radek in radky)
    {
        Console.WriteLine(radek);
    }
}
catch (Exception ex)
{
    Console.WriteLine("Chyba: " + ex.Message);
}
```

4. Výpis všech souborů a složek

```
void ProjdiSlozku(string cesta)
{
    foreach (string soubor in Directory.GetFiles(cesta))
```

```

    {
        FileInfo info = new FileInfo(soubor);
        Console.WriteLine("Soubor: " + soubor + " (" + info.Length + "
B)");
    }

    foreach (string slozka in Directory.GetDirectories(cesta))
    {
        Console.WriteLine("Složka: " + slozka);
        ProjdiSlozku(slozka);
    }
}

ProjdiSlozku("Data");

```

5. Spočítání souborů a složek

```

int pocetSouboru = 0;
int pocetSlozek = 0;

void Spocitej(string cesta)
{
    pocetSouboru += Directory.GetFiles(cesta).Length;
    pocetSlozek += Directory.GetDirectories(cesta).Length;

    foreach (string slozka in Directory.GetDirectories(cesta))
    {
        Spocitej(slozka);
    }
}

Spocitej("Data");

Console.WriteLine("Počet souborů: " + pocetSouboru);
Console.WriteLine("Počet složek: " + pocetSlozek);

```

6. Vypsání všech .txt souborů v adresáři

```

string[] soubory = Directory.GetFiles("Data", "*.txt");

foreach (string soubor in soubory)
{
    Console.WriteLine("Soubor: " + soubor);
    Console.WriteLine(File.ReadAllText(soubor));
}

```

7. Zápis jmen do souboru

```
string[] jmena = { "Jan", "Eva", "Petr" };  
  
Directory.CreateDirectory("Output");  
  
string cesta = Path.Combine("Output", "jmena.txt");  
  
File.WriteAllLines(cesta, jmena);
```